

02285 AI and MAS, F26
Exercises for week 5, 3/3-26
SOLUTIONS

Exercise 1 (Iterated Width in a simple domain)

Consider the planning domain given by the following set of actions $\mathcal{A}$:

ACTION a_1
PRECONDITION p_1
EFFECT p_2

ACTION a_2
PRECONDITION p_1
EFFECT $p_2 \wedge p_3$

ACTION a_3
PRECONDITION $p_1 \wedge p_2$
EFFECT p_4

ACTION a_4
PRECONDITION $p_1 \wedge p_3$
EFFECT p_5

ACTION a_5
PRECONDITION $p_1 \wedge p_4$
EFFECT p_5

ACTION a_6
PRECONDITION $p_4 \wedge p_5$
EFFECT p_6

- a) What is the set of ground atoms At in this planning domain? What is the size of T^1 ? And the size of T^6 ? Explain why $|T^6|$ (the size of T^6) is an upper bound on the size of the state space of any planning problem over $\mathcal{A}$.

SOLUTION. We have $At = \{p_1, \dots, p_6\}$. Size of T^1 is number of atoms plus 1, so 7. Size of T^7 is the number of substates of At , so $2^{|At|} = 2^6 = 64$. This is an upper bound on the size of

the state space, since no state can have size higher than 6, as there are only 6 atoms, and all real states are hence also substates in T^6 .

- b) Let $s_0 = \{p_1\}$, and define for each $i = 1, \dots, 6$ the planning problem $P(p_i) = (\mathcal{A}, s_0, p_i)$. Show that all $P(p_i)$ have solutions.

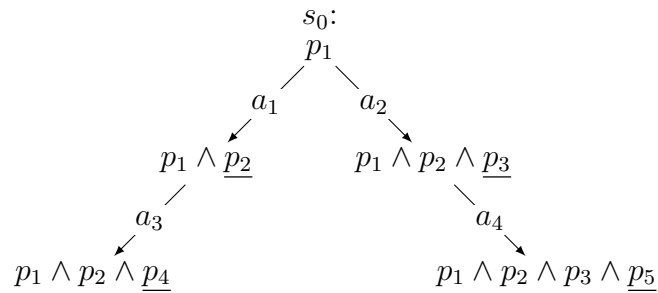
SOLUTION. We already have p_1 initially, and to get p_2 , we need to only execute a_1 . To get p_3 , we execute a_2 . If we execute both in sequence, we get the state $\{p_1, p_2, p_3\}$. From here, we can execute a_3 and a_4 in sequence to get to the state $\{p_1, \dots, p_5\}$. Finally, we can then execute a_6 to get also p_6 . In other words, executing the actions $a_1, \dots, a_4, a_6$ in sequence will lead to the state $\{p_1, \dots, p_6\}$, which contains all p_i . Hence all $P(p_i)$ have solutions.

- c) Now run IW(1) on each of the planning problems $P(p_i)$ (it will suffice to draw one search tree for all of them, as they only differ in the goal), using the following conventions:
- (a) When expanding a state, we always add the child nodes to the frontier in order according to the index of the action that generated the child, i.e., we always consider the actions in order $a_1, a_2, a_3, \dots$.
 - (b) In the BFS expansion of the tree, we always expand nodes from left to right.

Which of the planning problems are solved by IW(1), and why?

SOLUTION.

First note that whenever an action a_i has occurred in the search tree of IW(1), it can never occur again, as it would then only add the same atoms that it added the first time it was applied. We get the following search tree to depth 2 when running IW(1), where we have underlined a novel atom of each state:

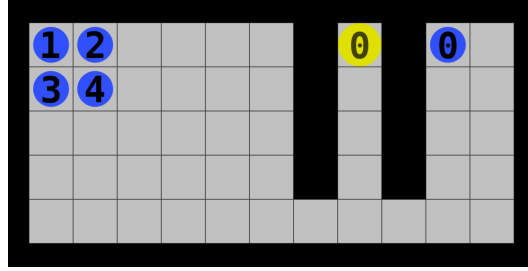


Since we can never reapply actions, a_1 and a_2 are blocked in the state $p_1 \wedge p_2$, and a_1 , a_2 and a_3 are blocked in the state $p_1 \wedge p_2 \wedge p_3$. Thus also in $p_1 \wedge p_2 \wedge p_4$ (left leaf), none of the action $a_1, \dots, a_4$ can be applied. Also a_5 cannot be applied, since p_5 already exists in the tree. And finally a_6 also cannot be applied, since its precondition isn't satisfied. Thus the state $p_1 \wedge p_2 \wedge p_4$ has no children. Looking at the state $p_1 \wedge p_2 \wedge p_3 \wedge p_5$ (right leaf), we similarly have that none of the actions $a_1, \dots, a_4$ can be applied, and the remaining two actions don't have their preconditions satisfied. Thus the tree cannot be expanded further.

We can therefore conclude that IW(1) solves $P(p_i)$ for all $i \leq 5$, but not for $i = 6$. The intuitive reason that $P(p_6)$ cannot be solved is that a_6 requires p_5 to be true, but there are two distinct ways to achieve p_5 (by a_4 or a_5), and we first explore achieving it by a_4 , but then that doesn't produce p_4 that we also need for a_6 to be applicable.

Exercise 2 (Iterated Width in MAPF)

Consider the following MAPF problem:



We can assume that states are described as sets of atoms of the form $AgentAt(agent, loc)$, where $agent$ is the agent number (0–4) and loc is the location of the agent (e.g. represented as a 2D coordinate).

- a) What are the optimal solutions to this problem? How long are the optimal solutions?

SOLUTION. All solutions where agent 0 moves around the wall along the shortest path to the goal, and is not obstructed by any of the other agents on its path. Optimal solutions have length 10.

- b) What will happen when running Iterated Width on this level? What is the lowest value of i for which $IW(i)$ will return a solution? (You should not actually run the algorithm by hand, but try to reason about what it will do.)

SOLUTION. For each joint move in $IW(1)$, the algorithm will check whether an agent enters some cell c for the first time. If not, no child will be added. In all optimal solutions, 0 follows an optimal path $c_0, \dots, c_n$ to the goal (where $n = 10$, cf. the first question). We show (by induction) that at least one of these optimal solutions will not be pruned: Let s_i denote the leftmost state at depth i of the search tree in which agent 0 is at c_i . When expanding s_i , we produce a new leftmost state s_{i+1} where agent 0 is at c_{i+1} for the first time, so this state is not pruned. Thus $IW(1)$ finds an optimal solution.

- c) Is the solution returned optimal?

SOLUTION. Yes, by the argument given in the previous answer.

- d) How large is the state space of the problem?

SOLUTION. Each agent can be in one of the $(6 + 1 + 2) \cdot 5 + 2 = 47$ cells, so the state space has size $47 * 46 * 45 * 44 * 43 = 184,072,680 \approx 2 \cdot 10^8$.

- e) How large is T^1 ?

SOLUTION. There are 47 locations and 5 agents, so there are $1 + 47 \cdot 5 = 236$ substates of size ≤ 1 .

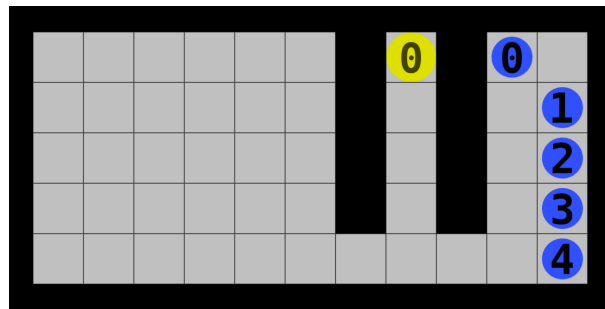
- f) Approximately how many distinct states will be generated by IW before it finds a solution?

SOLUTION. For $IW(0)$, only 1 state is generated. For $IW(1)$, we generate at most as many states as the size of T^1 , so at most 236. So in total at most 237 states.

- g) Approximately how many distinct states would greedy best-first graph search generate on this problem when using the Manhattan distance heuristic from agent to goal? Compare this to your number from the previous question.

SOLUTION. As long as the Manhattan distance of agent 0 can only increase, we will let all other agents move to all possible positions before considering to move agent 0 (we are then in a heuristic plateau). With 4 agents and 46 positions (excluding the position occupied by 0), this gives $46 \cdot 45 \cdot 44 \cdot 43 = 3.916.440$ states for each position of 0. We compute this many states for each position of 0 for which the Manhattan distance is at most 5 and for which agent 0 hasn't yet reached the corner of the wall (since to reach the corner of the wall, agent 0 will have to increase the Manhattan distance to 6). This amounts to 7 possible positions, so in total $7 \cdot 3.916.440 = 27.415.080$ states. This is a lower bound, as the solution hasn't yet been found, but from here, the Manhattan distance will go down, and the solution will be found relatively quickly. The 27+ million states computed by greedy best-first search is in stark contrast to the maximally 237 state computed by IW .

- h) Consider now instead the following variant of the problem:



Suppose first that when expanding a node, we always first consider the joint actions where as many of the individual actions as possible are noops. What is then the lowest value of i for which $IW(i)$ will return an optimal solution.

SOLUTION. It is $i = 1$. If we first consider joint actions where as many as possible of the individual actions are noops, we will consider agent 0 moving down while no other agent moves before we consider joint actions where also other agents move. This means that at depth 1 of the search tree, the leftmost state where agent 0 has moved one cell down, will be the one in which *only* agent 0 has moved. This allows us to expand that state with agent 0 moving one more cell down. Again, the leftmost of these will let the other agents do noops. In other words, the optimal solution where agent 0 moves along the optimal path and nobody else moves will not be pruned in $IW(1)$.

- i) What is the lowest value of i for which $IW(i)$ is guaranteed to return an optimal solution to this level, no matter in which order we add children to the search tree?

SOLUTION. We risk that when 0 follows its optimal path $c_0, \dots, c_n$, it will at some point be obstructed by another agent. Consider $IW(1)$ and suppose that at some depth i , the leftmost state s_i that has agent 0 at c_i is one in which cell c_{i+1} is blocked by another agent. As s_i might be the only state at depth i that has agent 0 at c_i and is not pruned, we might not reach a state at depth $i + 1$ with agent 0 at c_{i+1} . Hence $IW(1)$ might not find an optimal solution. $IW(2)$ necessarily finds optimal solution, however: If a depth of the search tree contains a state where 0 is at c_i and another agent j is at c_{i+1} , then there is also another unpruned state where j is at some other cell.

- j) Let i be the value from your previous question. Approximately how many distinct states does $IW(i)$ generate to find the optimal solution?

SOLUTION. As $i = 2$, we have substates of size ≤ 2 . We already know there were 237 substates of size ≤ 1 . For substates of size 2, we first choose an agent and a position for it, and then another agent and another position. This can be done in $5 \cdot 47 \cdot 4 \cdot 46 = 43.240$ different ways. So an upper bound on the number of generated states is $237 + 43.240 = 43.477$. It is still fairly impressive to solve a problem optimally in less than 50.000 states when the state space has size $2 \cdot 10^8$.

Exercise 3 (Repeated actions in IW)

- a) Consider the search tree for $IW(1)$ that you produced in Exercise 1. Note that each action a_i appear at most once in the tree. Now argue that for *any* planning problem, the search tree of $IW(1)$ will contain each (ground) action at most once, i.e., there will be at most one edge labelled with that action.

SOLUTION. We essentially already gave the required argument in our solution to Exercise 1: Any given ground action has a unique set of positive effects, so reapplying it can never produce any novel individual atoms.

- b) An action b is called the *inverse* of an action a , if for any state s in which a is applicable, executing a followed by b in s leads back to s , i.e., if

$$\text{RESULT}(\text{RESULT}(s, a), b) = s.$$

When the inverse of a exists, we usually denote it by a^{-1} . Note that each action in the hospital domain has an inverse. For instance, the inverse of $\text{Move}(N)$ is $\text{Move}(S)$.

Now prove that for any planning problem with inverses, if the search tree of $IW(1)$ has an edge labelled by an action a , then no other edge is labelled by a or a^{-1} . You are here encouraged to try to give a detailed mathematical proof.

SOLUTION. We already showed in the previous question that there can only be one a -edge in each tree. So we only need to consider the case of a^{-1} . However, as we are here asked to give a “proof”, not just an “argument”, let’s try to be a bit more formal about it. So suppose the tree contains an edge from s to t labelled a . To achieve a contradiction, suppose further that at a later stage another edge from s' to t' labelled b where $b = a$ or $b = a^{-1}$ was added. We

first consider the case where $b = a$. As usual, we let $\text{ADD}(a)$ denote the set of positive effects of a . Since a produced t when applied to s , we have $\text{ADD}(a) \subseteq t$ by definition of the semantics of STRIPS/PDDL (see the slides on Classical Automated Planning). Then we get, using again the semantics of PDDL, $t' = \text{RESULT}(s', a) = (s' - \text{DEL}(a)) \cup \text{ADD}(a) \subseteq s' \cup \text{ADD}(a) \subseteq s' \cup t$. As s' and t already existed in the tree before we added t' , we now have a contradiction, as all atoms of t' then already existed on the tree when t' was added. This completes the case $b = a$. Now consider the case $b = a^{-1}$. As in the previous case, we immediately get $\text{RESULT}(s', a^{-1}) \subseteq s' \cup \text{ADD}(a^{-1})$. Now we get $\text{ADD}(a^{-1}) \subseteq (t - \text{DEL}(a^{-1})) \cup \text{ADD}(a^{-1}) = \text{RESULT}(t, a^{-1}) = s$, since a^{-1} is the inverse of a , and a produced t from s . Putting these things together, we get $t' = \text{RESULT}(s', a^{-1}) \subseteq s' \cup \text{ADD}(a^{-1}) \subseteq s' \cup s$. However, now we again have a contradiction, as s' and s already existed in the tree when t' was added, and each atom of t' is either in s or s' . This completes the proof.

- c) It follows from the previous question that $IW(1)$ can never produce a solution where an action is performed twice, or performed once and then later inversed. Use this insight to design a level in the hospital domain that cannot be solved by $IW(1)$. Try to make the level as simple as possible.

SOLUTION. We can use the following level:



Note that there is an agent goal at the leftmost cell. Here the issue is that the agent needs to move backwards after having pushed the box to the goal, which is an inverse action, and hence not possible with $IW(1)$, according to the result of the previous question.

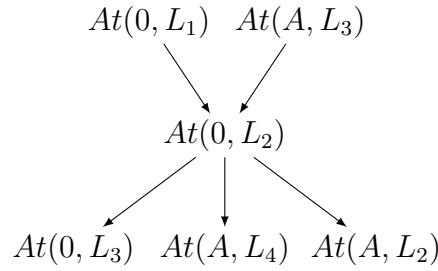
Note that we can't make the level even smaller as if we deleted the cell that is initially empty, then the first action would be a push and the second a move, so they are not each others inverse. Well, actually even that simpler level wouldn't be solvable by $IW(1)$, as the last move would not produce any new atoms, but then we can't argue using the result of the previous question any more.

- d) Will the level you designed in the previous question necessarily be solved by $SIW(1)$, i.e., SIW restricted to width 1? By "necessarily solved" we mean that it will be solved independently of the order in which subgoals are considered, independently of the order in which actions are considered during node expansion and independently of the order in which nodes at the same level of the tree are expanded. If the answer to the question is yes, then try to design another simple level that also $SIW(1)$ cannot necessarily solve.

SOLUTION. Actually, $SIW(1)$ does *not* necessarily solve our level. It might choose the agent goal first. That is solved by $IW(0)$, since it is already satisfied in the initial state. But then the algorithm is restarted with both subgoals, and attempted to be solved with $IW(1)$, which we from the previous question know is not possible.

Remark: Consider, though, if the subgoal order had been opposite, i.e., the first subgoal chosen was the box goal. That subgoal actually has width 1, which is easily seen by building (the

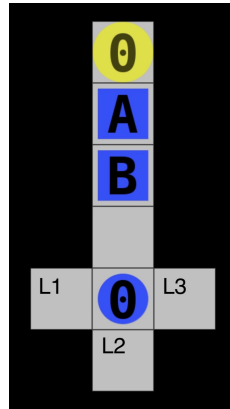
relevant part of) $\mathcal{G}^1$. More precisely, letting the cells of the level be named L_1 to L_4 from left to right, $\mathcal{G}^1$ is (ignoring rigids):



The box goal is $At(A, L_4)$, and since $\mathcal{G}^1$ contains a state satisfying that atom, $\mathcal{G}^1$ contains a state that optimally implies it. This implies that the subgoal $At(A, L_4)$ has width 1, by definition. From one of the theorems on the slides, we then have that $IW(1)$ will solve $At(A, L_4)$ optimally. Hence, if we run SIW with the subgoal $At(A, L_4)$ considered first, we necessarily solve that subgoal at width 1. Afterwards, the algorithm will restart with both subgoals included, the box now already being at its goal. The new problem is also easily seen to have width 1, so will also be solved optimally. So with this goal order, SIW actually will solve the problem at width 1—and solve it optimally.

- e) Did the level you designed in the previous question have 1 or multiple subgoals? If multiple, can you design a level with a single subgoal that $IW(1)$ cannot solve?

SOLUTION. The following level works:



Here the agent needs to move A and B out of the way to reach the goal cell at the top. The first move is $Move(N)$, so the agent cannot later execute that same action from its initial position. This implies that after having put B aside in $L1$, $L2$ or $L3$ and being back in the initial position, the agent is forced to drag B with it to reach the A box. But then the B box cannot be pushed down again, as that would lead to no novel atoms. Hence it cannot be solved at width 1.

What we learn from this exercise is that in general we should not expect too much from $IW(1)$ in the hospital domain, since actions cannot be repeated or inversed, meaning that we for instance

cannot move back along a path that we earlier moved forward along (unless we then also carry a box one of the times). Such moving back and forth can be expected to happen quite regularly in single-agent levels with many boxes. Think for instance about how often a warehouse robot might use the same path. We should then at least move to $SIW(1)$ if we want something that works well in the hospital domain. However, as the last question shows, even $SIW(1)$ can be fooled. But now note that it never makes sense to do the exact same action twice or undo an earlier action unless something else has changed in the meantime (this is true for *any* domain, also in real life). That ‘something’ is not necessarily an explicit subgoal in the formulation of the problem, but it could easily be an implicit subgoal. For instance in the hospital domain, an implicit subgoal could be ‘move box out of path’, if a box is initially blocking a path that is required to achieve another (sub)goal. Adding such implicit subgoals as explicit subgoals can then actually help to make SIW perform better, in particular if the subgoals are also considered in a clever order. Note the relation to HTN: One could consider implicit subgoals such as ‘move box out of path’ as high-level actions, and then first finding a high-level plan using actions such as ‘move to box’, ‘move box out of path’ and ‘move box to goal’ can help significantly in reducing the search efforts.

Exercise 4 (Improving IW?)

Above we saw that the order of adding children can matter. Consider the following alternative to IW that tries to remedy this:

Definition. $IW'(i)$ is the algorithm that runs like breadth-first search except a child state s is only added to the frontier if it contains at least one new T^i state that is not already contained in any tree node at strictly lower depths of the tree.

- a) Will $IW'(1)$ find an optimal solution to any MAPF problem where only 1 agent has a goal, and the optimal path is initially not obstructed?

SOLUTION. Yes, now we don’t block any of the states at depth i where the agent is at c_i .

- b) $IW(i)$ has the property that the number of states it generates is at most exponential in i , more precisely, the number of distinct states it generates is at most $|T^i| \leq (|At| + 1)^i$. Is the same true for $IW'(i)$?

SOLUTION. For each substate T^i , we can build $2^{|At|-i}$ distinct “real” states by varying the values on the other atoms. In the worst case, each new depth of the tree built by $IW'(i)$ might only introduce one new bounded state T^i and still have size $2^{|At|-i}$. The upper bound on the number of states is then $|T^i| \cdot 2^{|At|-i} \leq (|At| + 1)^i 2^{|At|-i}$. This is unfortunately exponential in the number of atoms, where $IW(i)$ was exponential in i . So the answer is no.

Concretely, for the levels considered above, with $i = 1$, we have that even if agent 0 follows the optimal path to the goal, we then still allow the other agents to perform all their possible combinations of moves at the same depth. Suppose that initially agents 1–4 are in positions where they are free to move in all 4 directions. Then they each have a branching factor of 5. So they can in one joint move generate $5^4 = 625$ possible configurations. This means that already at depth 1, $IW'(1)$ would generate 625 states. This is in contrast to $IW(1)$, that generates in total at most $(|At| + 1)^1 = 236$ states in the entire tree.

Exercise 5 (OPTIONAL) (Proof of correctness and completeness of IW)

The slides stated the following theorem.

Theorem. Let P be a planning problem of width w . Then $IW(w)$ solves it optimally.

Prove the theorem. *Hint:* Part of the proof will be by induction on the depth of the search tree.

SOLUTION. There is a path $t_0, \dots, t_k$ in $\mathcal{G}^w$ from a root vertex to a vertex t_k optimally implying the goal g . By choice of path and by definition of $\mathcal{G}^w$, each planning problem $P(t_i)$ then has an optimal solution of length i .

We show that for all $0 \leq i \leq k$, there exists at least one state s_i at depth i of the search tree of $IW(w)$ containing t_i . By induction on i . Suppose there is a state s_i at depth i containing t_i (induction hypothesis). Then t_i is reached by an optimal plan for $P(t_i)$. Since there is an edge $(t_i, t_{i+1}) \in \mathcal{G}^w$, the optimal plan for $P(t_i)$ can be extended with an action a to give an optimal plan for $P(t_{i+1})$. By expanding s_i with a we hence get a child state s_{i+1} at depth $i+1$ containing t_{i+1} (unless we are blocked from adding s_{i+1} at depth $i+1$, but that can only happen if a state containing t_{i+1} already exists at depth $i+1$, and then we're also done).

We now have a path $s_0, \dots, s_k$ in the search tree that is optimal for $P(t_k)$, and since t_k optimally implies g , it is also optimal for $P(g) = P$.